

CSP初赛集训
第10课
数据结构 线性结构

2023.07.27

01 数据结构概念



程序=算法+数据结构



数据结构三要素

数据结构有三个要素：

- **逻辑结构**
- **物理结构**
- **数据的运算**

逻辑结构

逻辑结构是指数据元素之间的逻辑关系。

它更贴近于现实，即从逻辑关系上来描述数据。

它是独立于计算机的，与计算机内部如何存储是无关的。

线性结构: 线性表，栈，队列，双队列，串

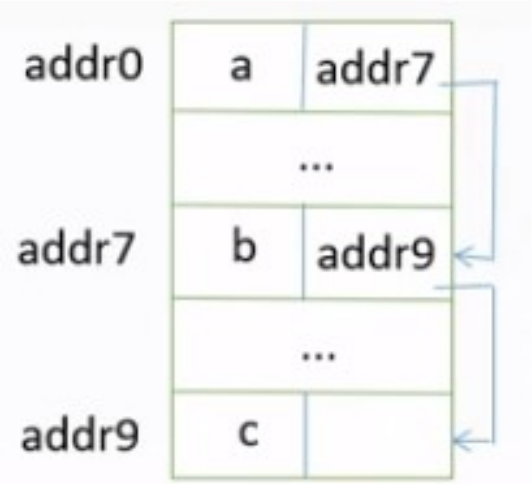
非线性结构：集合、高维数组，树形结构、图状结构

物理结构

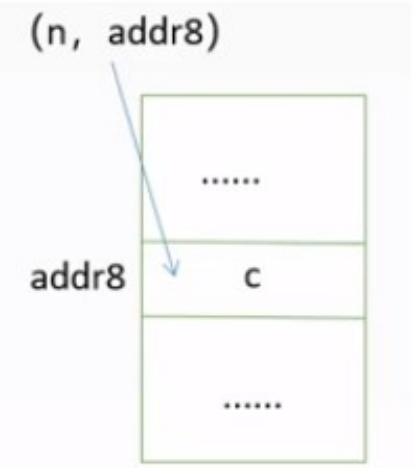
物理结构:是指数据结构在计算机内的表示，也称为**存储结构**。



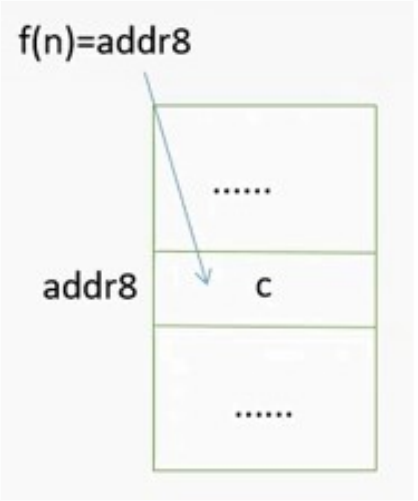
顺序存储



链式存储



索引存储



散列存储

02 线性表

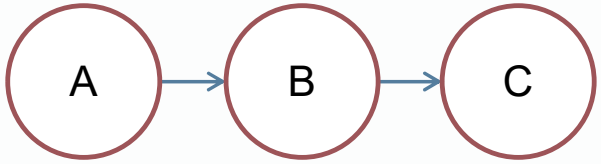


线性表

线性表：是n个具有相同特性的数据元素的有限序列。

线性表中数据元素之间的关系是一对一的关系，即除了第一个和最后一个数据元素之外，其它数据元素都有且只有一个前趋元素，有且只有一个后继元素。

元素
A
B
C

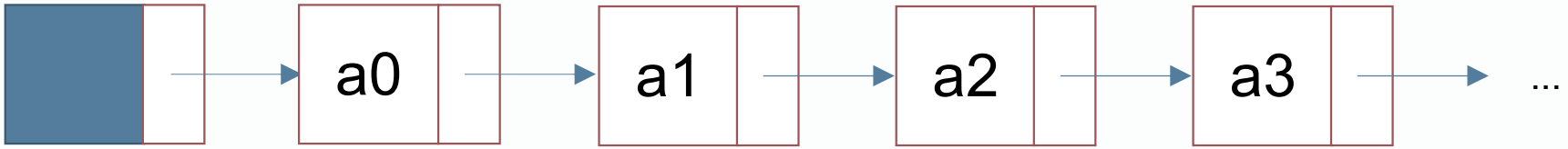


线性表与链表

顺序存储结构的线性表，也叫顺序表。

链式存储结构的线性表，也叫链表。

链表有头结点。



顺序表

顺序表：顺序存储结构的线性表

```
DataType a[N];
int len;
```

- DataType：数据类型。
- N: 最大可能存储的数据条数
- len：顺序表中数据个数

字符串是数据类型为char的顺序表。

练习

[J-2012-19]原字符串中任意一段**连续的字符**所组成的新字符串称为**子串**。则字符“AAABBBCCC”共有（ ）个不同的非空子串。

A. 3

B. 12

C. 36

D. 45

正确答案： C

单向链表

```
struct Node
{
    DataType val;//DataType:数据类型 val: 数据的值
    Node* next; //下一个结点的地址
};
```

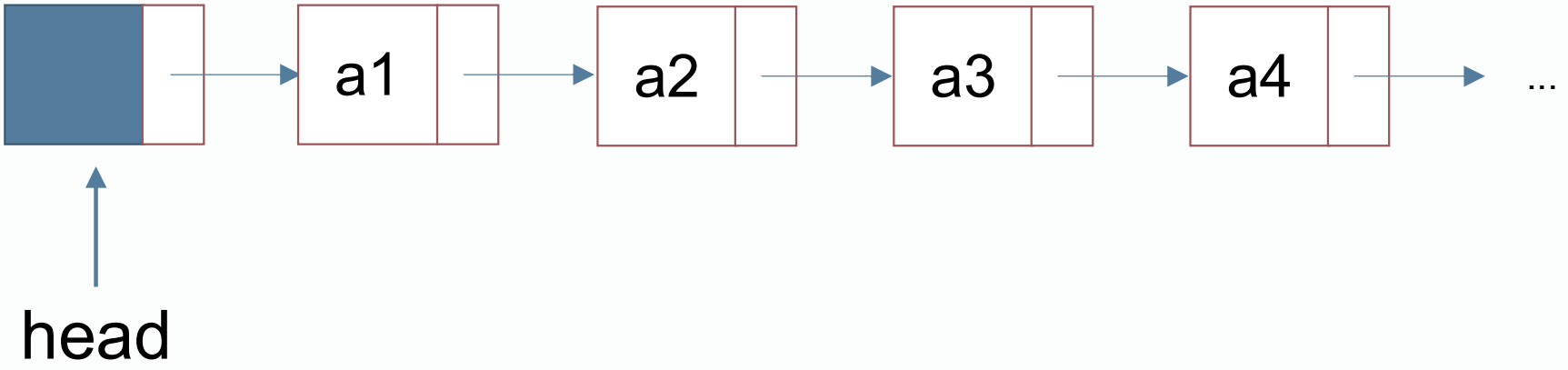
从堆区内存申请内存，地址可以不连续。

```
Node * np = new Node;
```



头结点

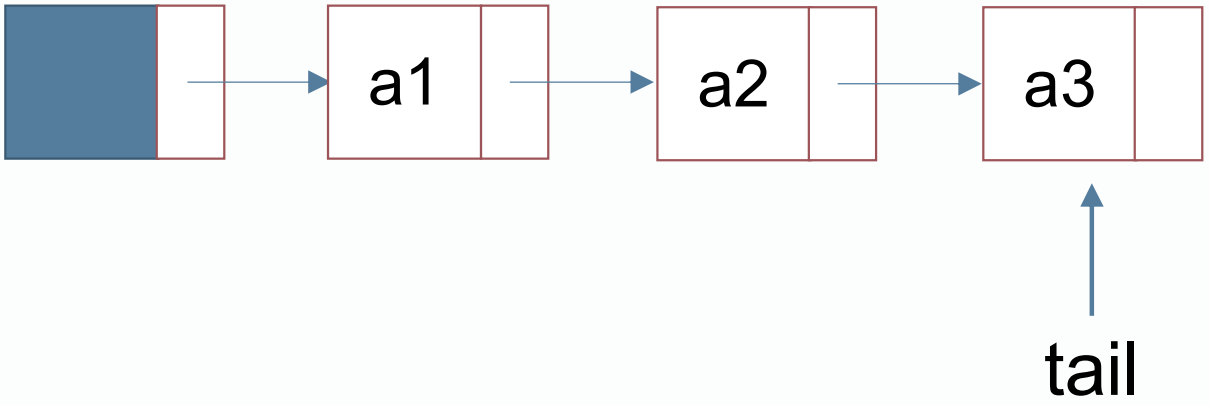
头结点：该结点无数据，其next指向链表的第一个结点




```
Node* head = new Node;
```

尾指针

尾指针：指向链表的最后一个结点

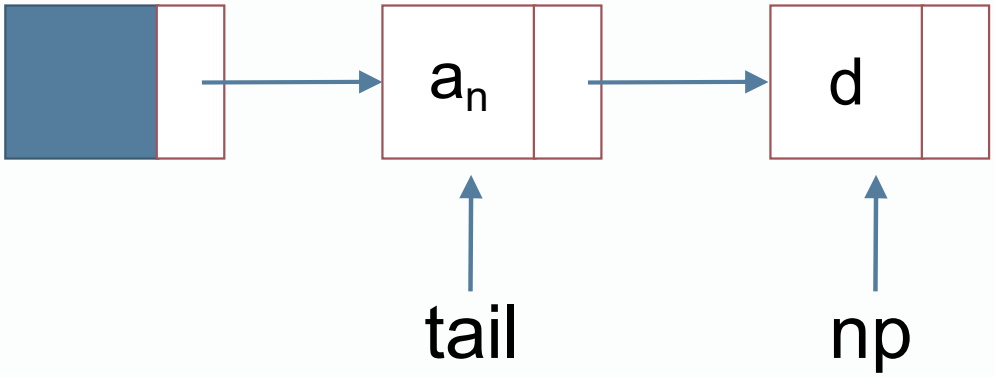



```
Node* tail = head; //初始情况下尾指针指向头结点
```

链表的运算

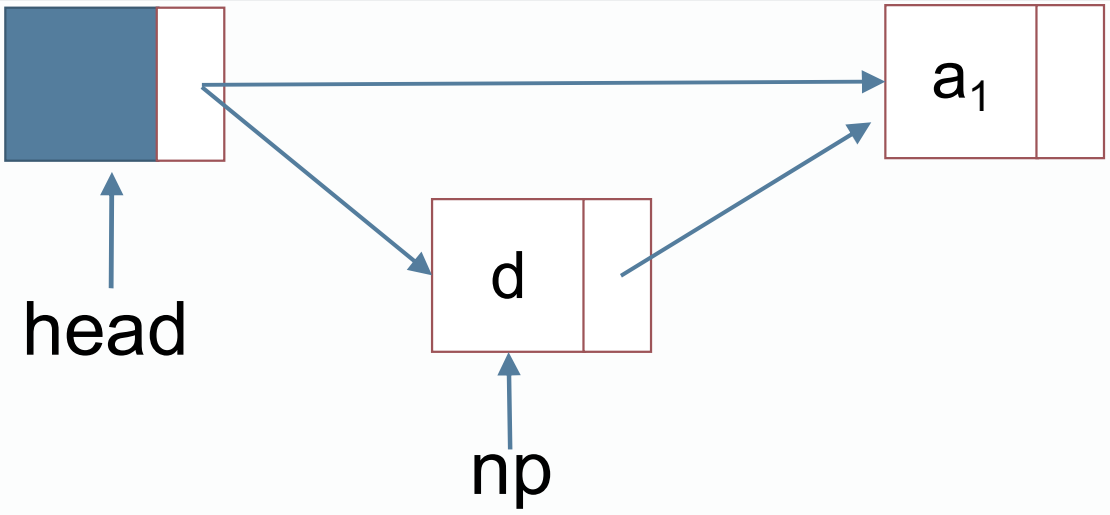
数据运算	函数声明
返回第一个元素的值	*DataType* front();
头插法添加数据	void push_front(*DataType* d);
尾插法添加数据	void push_back(*DataType* d);
获取第i项数据的结点地址	int getAddr(int i);
在t指向的结点**后**插入数据	void insert(int t, *DataType* d);
删除t指向结点的**下一个**结点	void erase(int t);
修改t指向的结点的数据	void set(int t, *DataType* d);
获取t指向的结点的数据	*DataType* getVal(int t);

尾插法添加结点 push_back



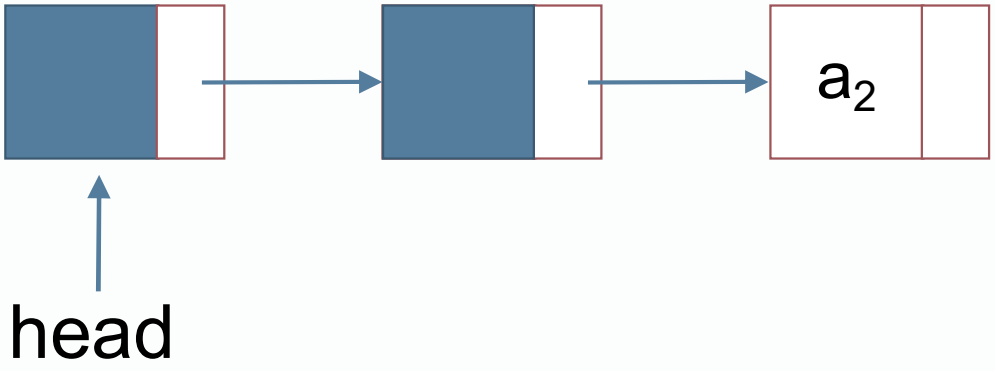

```
Node* np = new Node;
np->val = d;
tail->next = np;
tail = np;
```

头插法添加结点 push_front



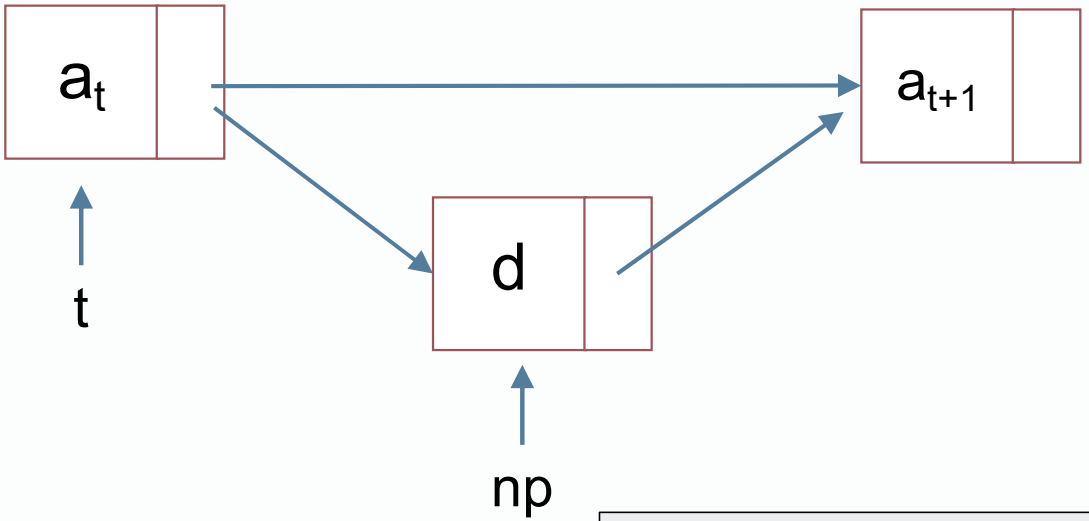

```
Node* np = new Node;
np->val = d;
np->next = head->next
head->next = np
```

删除第一个结点 pop_front



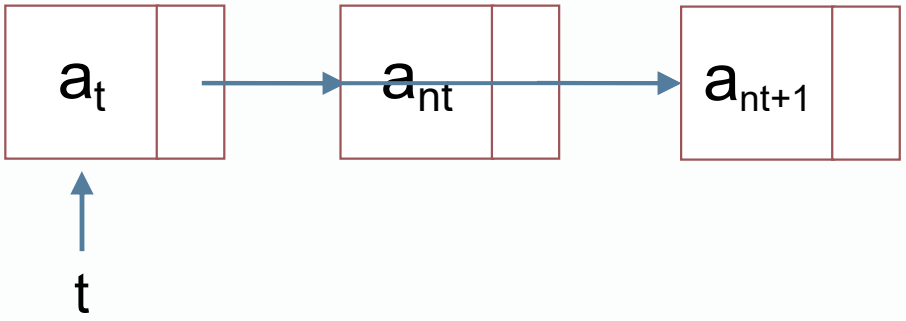

```
head = head->next
```

链表插入 insert_after（在t指向的结点后插入）




```
Node* np = new Node;
np->val = d;
np->next = t->next;
t->next = np;
```

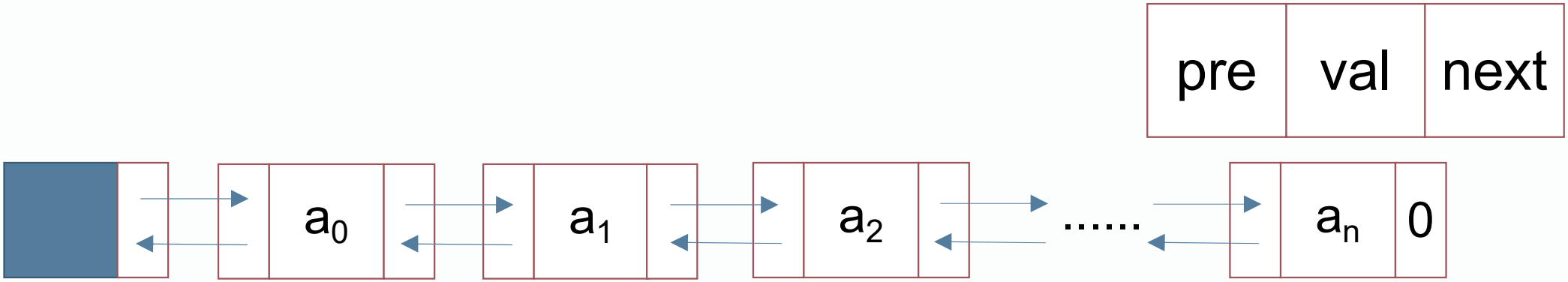
链表删除 erase_after（删除t指向的结点的下一个结点）




```
t->next = t->next->next;
```

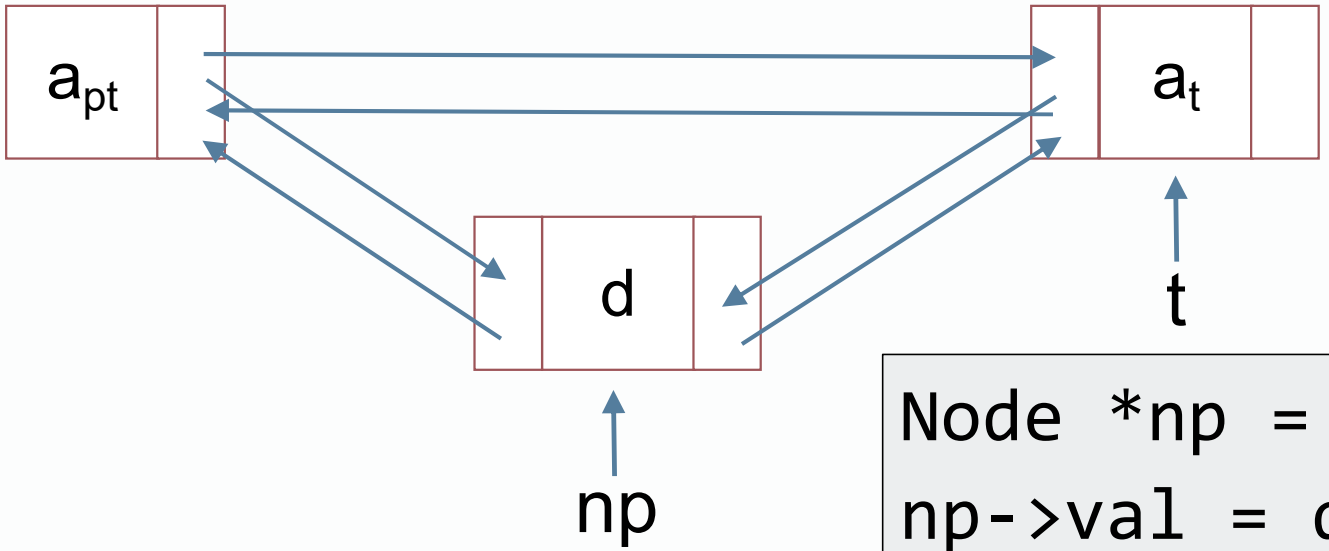
双向链表

```
struct Node
{
    DataType val;//DataType:数据类型
    Node *pre, *next; //pre:前 next:后
};
```



链表插入

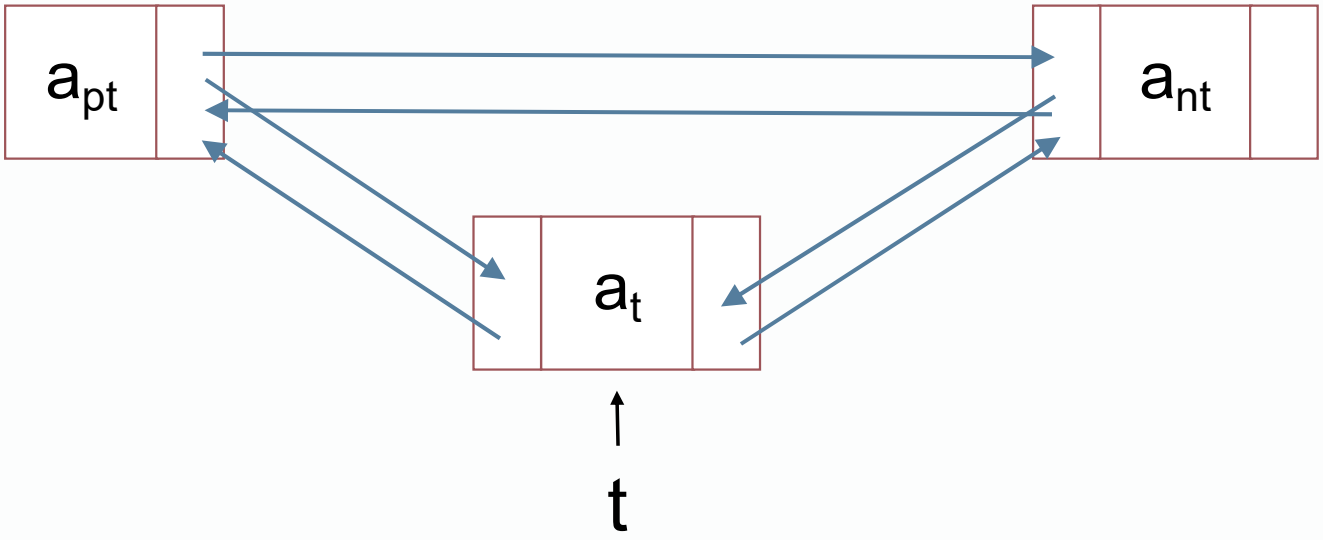
在t指向的结点前插入，返回插入的新结点地址




```
Node *np = new Node;
np->val = d;
np->next = t;
np->pre = t->pre;
t->pre->next = np;
t->pre = np;
```

链表删除

删除t指向的结点，返回t的下一个结点地址




```
t->pre->next = t->next;
t->next->pre = t->pre;
```

线性表的两种存储结构顺序表与链表

顺序表：

- 优点：查找速度快，存取高效
- 缺点：

插入和删除比较慢

元素地址必须连续

链表：

- 优点：插入和删除速度快
- 元素地址可以不连续
- 缺点：

查找速度慢，需要遍历链表

练习

[J-2020-7][J-2019-6][J-2015-13][J-2014-10]

链表不具有的特点是（）。

A. 可随机访问任一元素

B. 不必事先估计存储空间

C. 插入删除不需要移动元素

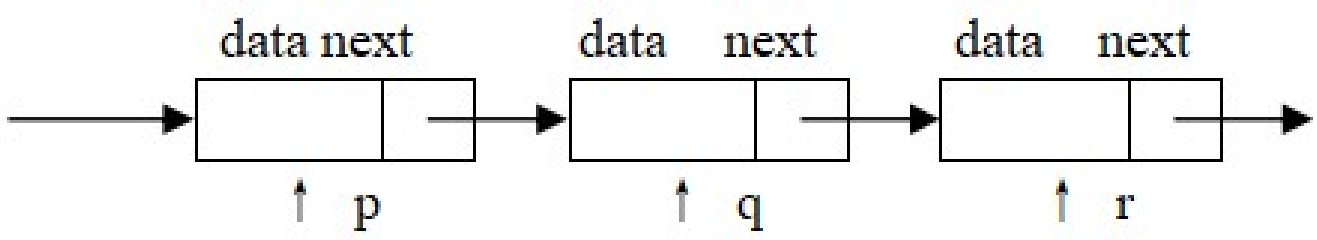
D. 所需空间与线性表长度成正比

正确答案： A

练习

[S-2014-11]有以下结构体说明和变量定义，如图所示，指针 p,q,r 分别指向一个链表中的三个续结点。

```
struct node {
 int data;
 struct node *next;
} *p,*q,*r;
```



现要将 q 和 r 所指结点的先后位置交换，同时要保持链表的连续，以下程序段中错误的是( )

A. q->next = r->next; p-> next = r; r->next = q;

B. p->next = r; q->next = r->next; r->next = q;

C. q->next = r->next; r->next = q; p->next = r;

D. r->next = q; q->next = r->next; p->next = r;

正确答案： D

栈的逻辑结构

栈(stack)：是一种受限定的线性表，限定其插入和删除操作只能在表的一端进行。

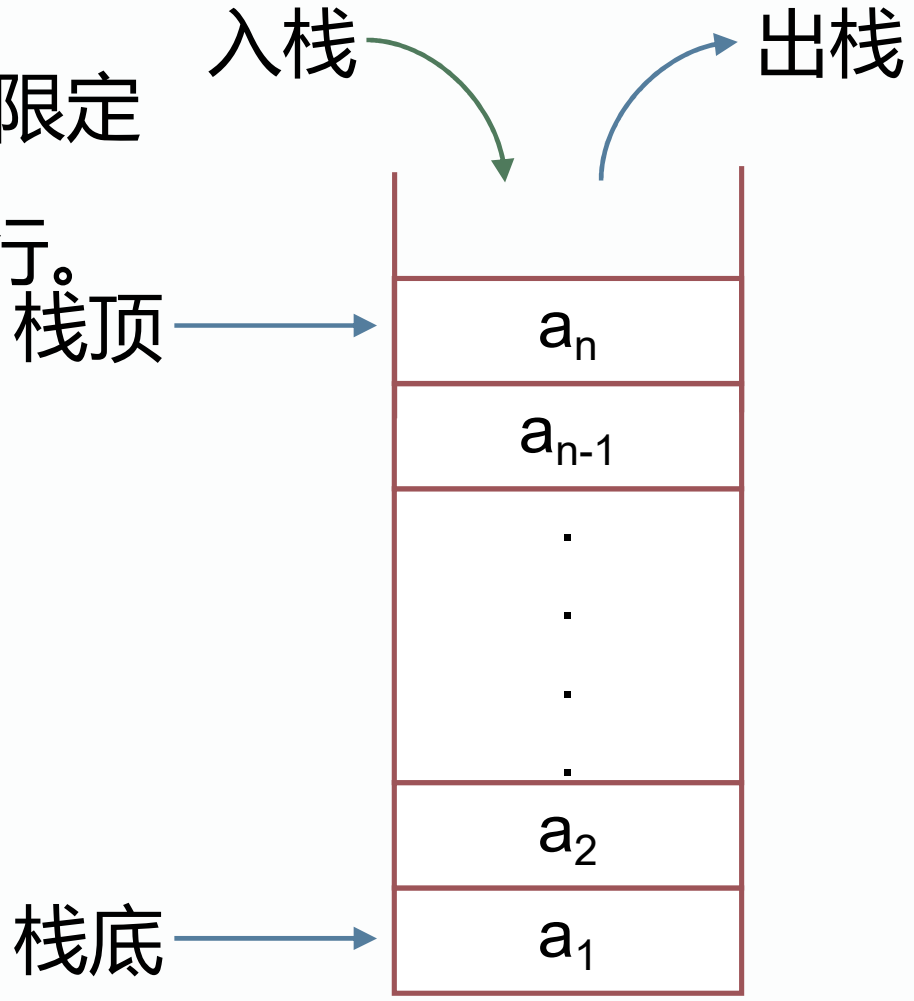
栈顶：允许操作的一端

栈底：不允许操作的一端

入栈 / 压栈 / 进栈(push)：添加数据

出栈 / 弹栈 / 退栈(pop)：删除数据

栈的特点：后进先出



栈的实现

顺序栈：

```
DataType stk[N];
int tp; //栈顶位置
```

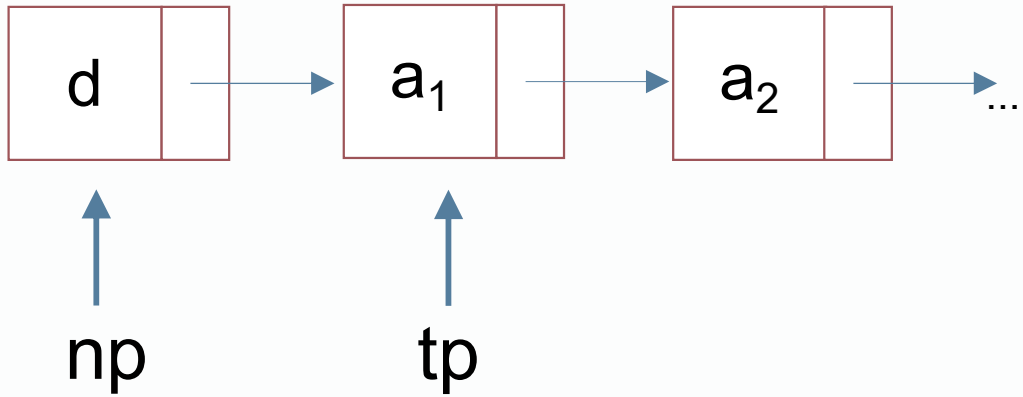
链栈：

```
struct Node
{
    DataType val;
    Node *next;
};
Node *tp;//栈顶指针
```

链式存储结构栈

- 入栈（压栈）

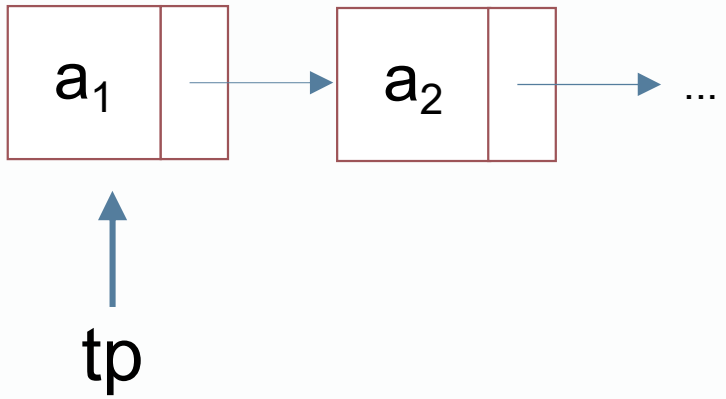
```
void push(DataType d)
{
    Node* np = new Node;
    np->val = d;
    np->next = tp;
    tp = np;
}
```



链式存储结构栈

- 出栈（弹栈）

```
void pop()
{
    tp = tp->nexts;
}
```



C++ STL stack

C++ STL stack用法	头文件：<stack>
stack<*DataType*> stk;	栈初始化，<>中填栈中元素的类型
stk.push(a);	将a入栈
stk.empty();	栈是否为空
stk.top();	返回栈顶元素（如栈空，则会RE）
stk.pop();	出栈（不返回值）
stk.size();	返回元素个数
stk = stack<*DataType*>();	清空栈

练习

[J-2017-16]对于入栈顺序为 a, b, c, d, e, f, g 的序列，下列（ ）不可能是合法的出栈序列。

A. a, b, c, d, e, f, g

B. a, d, c, b, e, g, f

C. a, d, b, c, g, f, e

D. g, f, e, d, c, b, a

正确答案： C

队列的逻辑结构

队列(queue)：是一种特殊的线性表，仅允许在一端插入，而在另一端删除。

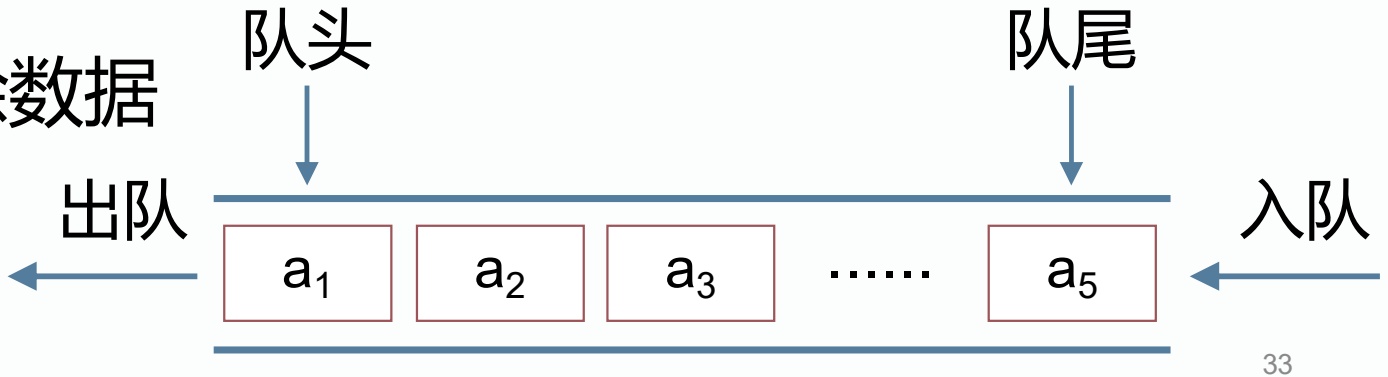
队尾(tail)：允许插入数据的一端

队头(head)：允许删除数据的一端

入队(enqueue)：插入数据

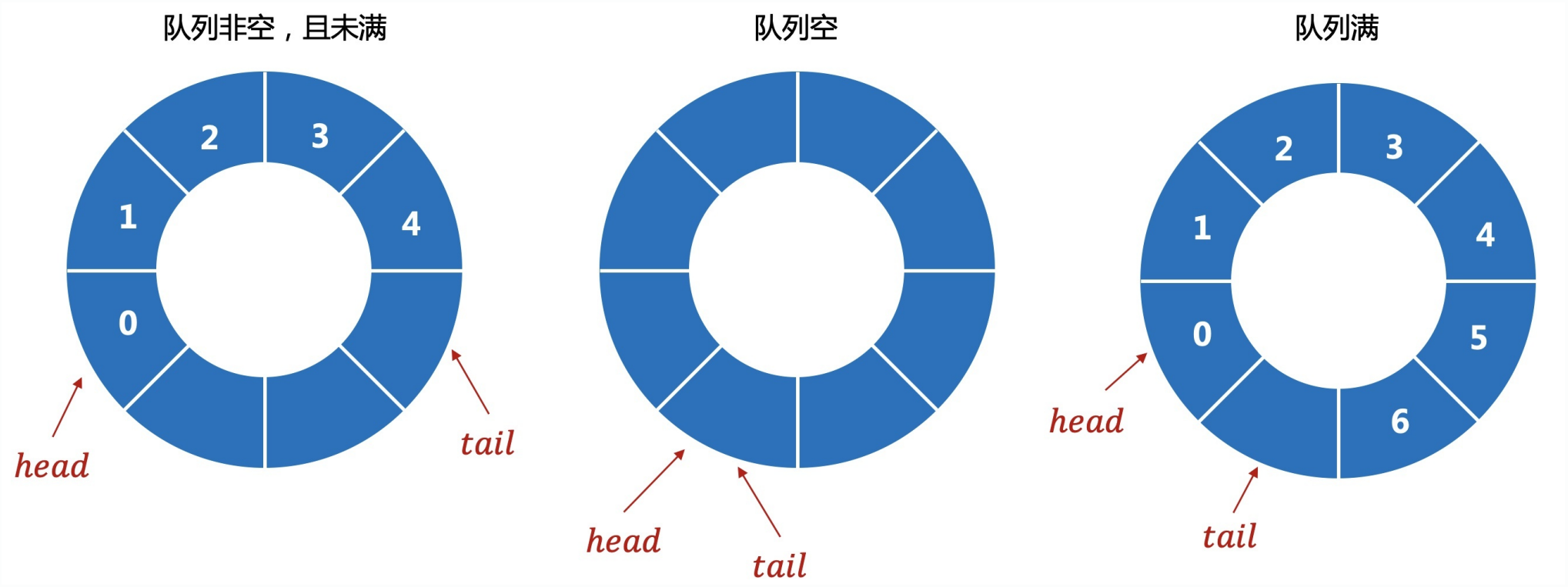
出队(dequeue)：删除数据

队列的特点：先进先出



循环队列

循环队列就是将队列存储空间的最后一个位置绕到第一个位置，形成逻辑上的环状空间，供队列循环使用。



循环队列

N：循环队列数据最大容量（数组长度）

循环队列中取i的下一个位置(i+1)%N

循环队列中取i的前一个位置(i+1+N)%N

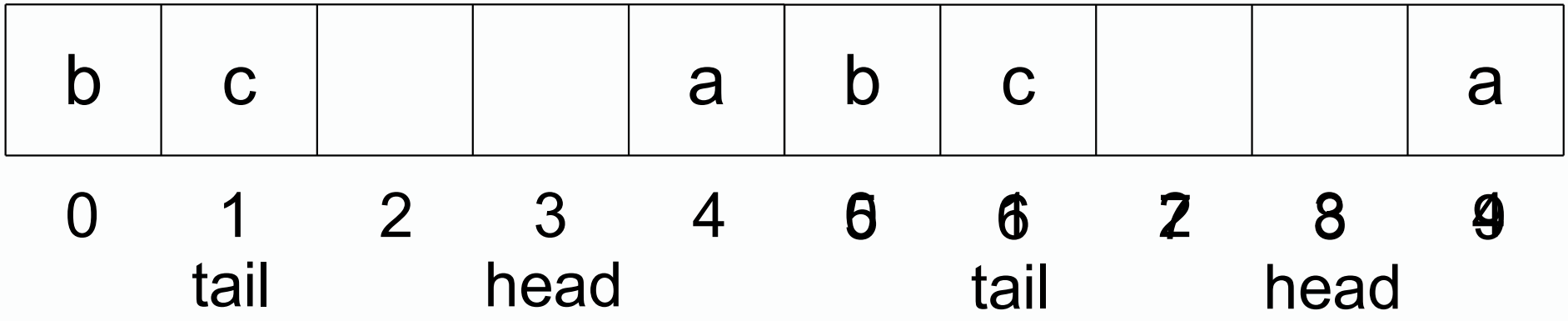
循环队列为空：tail == head

循环队列已满： (tail+1)%N == head

循环队列

- 求循环队列中元素个数

```
int size()
{
    return (tail + N - head) % N;
}
```



队列

现有一循环队列，其队头指针为front，队尾指针为rear，循环队列长度为N,最多存储N-1个数据。其队内有效长度为（）

A.(rear - front + N) % N + 1

B.(rear - front + N) % N

C.(rear - front) % (N + 1)

D.(rear - front + N) % (N - 1)

正确答案：B

C++ STL

C++ STL queue用法	
queue<int> que;	队列初始化，<>中填队中元素的类型
que.push(a);	将a入队
que.pop();	删除队头元素（无返回值）
que.empty();	判断队列是否为空
que.front();	返回队头元素
que.back();	返回队尾元素
que.size();	返回元素个数

表达式求值问题

中缀表达式：运算符位于操作数之间。

例：(3+4)×5-6

前缀表达式（波兰表达式）：运算符位于操作数之前。

例：-×+3456

后缀表达式（逆波兰表达式）：运算符位于操作数之后。

例：34+5×6-

练习

[J-2017-12]表达式 a * (b + c) * d 的后缀形式是

A. a b c d * + *

B. a b c + * d *

C. a * b c + * d

D. b + c * a * d

正确答案： B

练习

[J-2010-9]前缀表达式+ 3 * 2 + 5 12的值是（ ）。

A. 23

B. 25

C. 37

D. 65

正确答案： C



谢 谢